# Written Report for AI 3 Labwork 2

# Recommender Engine I

Technische Hochschule Ingolstadt

22.05.2026

Group 02 : Southik Banerjee, Divyesh Hauzaree, Mohamed Farag

# Table of Content

# 1. Task 1. Developing a simple Recommender engine using cosine similarity

## 1.1 Simple movie recommender program using Cosine Similarity

The program begins by loading the movie dataset with pandas and cleaning the data by removing extra spaces and missing values (ex : .strip()). It keeps only the required columns: Film, Genre, and Lead Studio. The genre and studio information are then combined into a new features column, which represents each movie's characteristics.

1. Load & Pre-process Data

```
df = pd.read_csv("movies-mod-sim-neu.csv")
df.columns = df.columns.str.strip()

# Keep only the three attributes specified by the task; drop rows with missing values
df = df[["Film", "Genre", "Lead Studio"]].dropna().reset_index(drop=True)

# Strip stray whitespace from all text columns
df["Film"]         = df["Film"].str.strip()
df["Genre"]        = df["Genre"].str.strip()
# Merge internal spaces in Lead Studio so "Warner Bros" → "WarnerBros" (one token)
df["Lead Studio"]  = df["Lead Studio"].str.strip().str.replace(r"\s+", "", regex=True)

print(f"Loaded {len(df)} movies\n")
print(df.to_string(index=True))
```

[3]

```
Loaded 60 movies

                                  Film                   Genre     Lead Studio
0   Tyler Perry's Why Did I get Married                 Romance     Independent
1               Twilight: Breaking Dawn                  Comedy     Independent
2                              Twilight                 Romance          Summit
3                       The Ugly Truth  Science Fiction Drama     Independent
4             The Time Traveler's Wife                   Drama       Paramount
5                         The Proposal          Romantic Comedy          Disney
6                 The Invention of Lying          Romantic Comedy       WarnerBros
7                     The Heartbreak Kid          Romantic Comedy       Paramount
8                            The Duchess                   Drama       Paramount
9   The Curious Case of Benjamin Button                 Fantasy       WarnerBros
10                    The Back-up Plan          Romantic Comedy             CBS
```

Next, Scikit-learn's CountVectorizer converts these text features into numerical vectors using a binary representation (0 or 1). The cosine_similarity function is then used to calculate how similar each movie is to every other movie. A similarity value closer to 1 means the movies are more alike.

## 2. Build Feature Matrix & Cosine Similarity Matrix

Each movie is represented by a binary token vector of its **Genre** words and **Lead Studio** token (Film title is the identifier, not a feature — see task Fig. 2).

```
    # Feature string: Genre tokens + Lead Studio token (already merged to one word)
    df["features"] = df["Genre"] + " " + df["Lead Studio"]

    # CountVectorizer with binary=True produces a 0/1 token-count matrix
    cv = CountVectorizer(binary=True)
    count_matrix = cv.fit_transform(df["features"])

    # Cosine similarity: value of 1.0 = identical feature vectors (best match)
    sim_matrix = cosine_similarity(count_matrix)

    print(f"Vocabulary ({len(cv.get_feature_names_out())} tokens):")
    print(sorted(cv.get_feature_names_out()))
    print(f"\nSimilarity matrix shape: {sim_matrix.shape}")
```

```
Vocabulary (37 tokens):
['20thcenturystudios', 'a14', 'action', 'adventure', 'animation', 'cbs', 'columbiapictures', 'comedy', 'crime', 'dama', 'disney', 'drama', 'erotic', '

Similarity matrix shape: (60, 60)
```

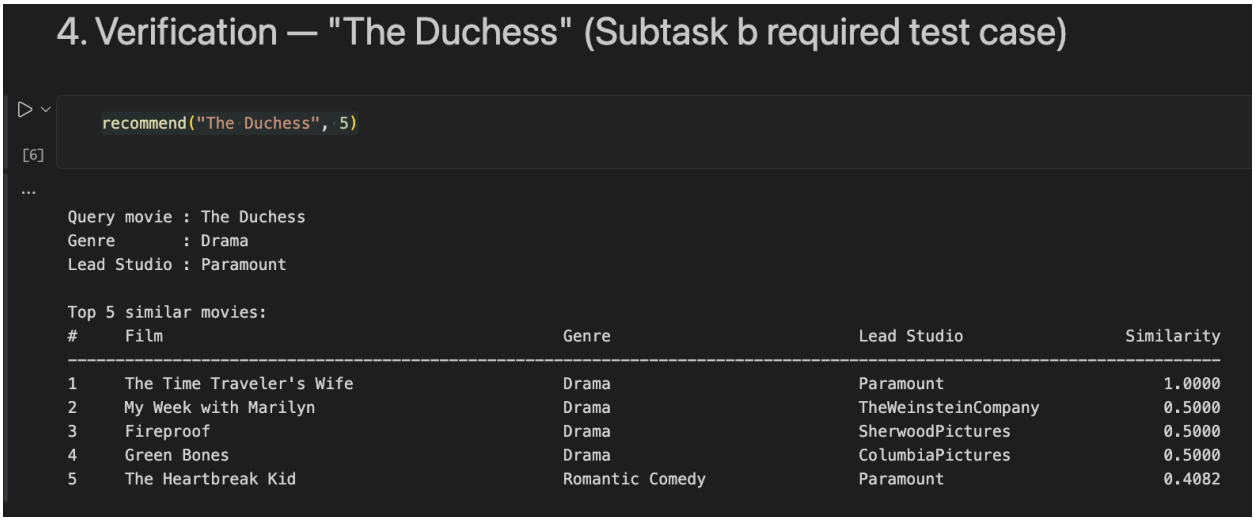
The recommend() function takes a movie title and amount of recommendations as input. It first checks whether the movie exists in the dataset, if found, the function retrieves the similarity scores, sorts them from high to low, and displays the most similar movies calculated using cosine similarity along with their genres, studios, and similarity values.

```
def recommend(movie_name: str, n: int) -> None:
    """
    Print the top-n most similar movies to `movie_name` using cosine similarity.
    Higher cosine similarity value = more similar (closer feature vectors).
    Gracefully handles movie names not present in the dataset.
    """
    matches = df[df["Film"].str.lower() == movie_name.strip().lower()]

    if matches.empty:
        print(f"Movie '{movie_name}' was not found in the dataset. Please check the spelling.")
        return

    idx = matches.index[0]
    query = df.loc[idx]
    print(f"\nQuery movie : {query['Film']}")
    print(f"Genre       : {query['Genre']}")
    print(f"Lead Studio : {query['Lead Studio']}")

    # Build sorted list of (index, similarity) excluding the query movie itself
    scores = sorted(
        [(i, sim_matrix[idx, i]) for i in range(len(df)) if i != idx],
        key=lambda x: x[1],
        reverse=True   # highest cosine similarity first = best match
    )

    col_w = {"rank": 5, "film": 45, "genre": 30, "studio": 25, "sim": 12}
    header = (f"{'#':<{col_w['rank']}} {'Film':<{col_w['film']}} "
              f"{'Genre':<{col_w['genre']}} {'Lead Studio':<{col_w['studio']}} "
              f"{'Similarity':>{col_w['sim']}}")
    print(f"\nTop {n} similar movies:")
    print(header)
    print("-" * len(header))

    for rank, (i, score) in enumerate(scores[:n], 1):
        row = df.loc[i]
        print(f"{rank:<{col_w['rank']}} {row['Film']:<{col_w['film']}} "
              f"{row['Genre']:<{col_w['genre']}} {row['Lead Studio']:<{col_w['studio']}} "
              f"{score:>{col_w['sim']}.4f}")
```

3. Recommender Function (Subtask a)

## 1.2 Subtask b) Verify the correctness of the recommender

The correctness of the recommender system can be verified by comparing the attributes of the recommended movies with the attributes of the entered movie. For example, when calling recommend("The Duchess", 5), the entered movie has the genre Drama and the lead studio Paramount. The recommended movies mostly share the same genre (Drama) and some also share the same lead studio (Paramount), which shows that the cosine similarity algorithm is correctly identifying movies with similar attributes.



Additional verification examples using different movie names, genres, and lead studios can also be found in the code outputs to further confirm the accuracy of the recommender system.

## 5. Additional Verification Examples

```
# Different genre + studio → expect different cluster of results
recommend("Spider-Man", 5)
recommend("Twilight", 5)
recommend("The Proposal", 5)
```

```
Query movie : Spider-Man
Genre       : Science Fiction Action
Lead Studio : ColumbiaPictures

Top 5 similar movies:
#     Film                                    Genre                       Lead Studio                Similarity
-------------------------------------------------------------------------------------------------------------------
1     Iron Man 3                              Action Science Fiction      ColumbiaPictures               1.0000
2     Infinite                                Science Fiction Action      ParamountPictures              0.7500
3     X-Men                                   Action Science Fiction      20thCenturyStudios             0.7500
4     Spider-Man 2                            Action                      ColumbiaPictures               0.7071
5     Avatar                                  Science Fiction             ParamountPictures              0.5774

Query movie : Twilight
Genre       : Romance
Lead Studio : Summit

Top 5 similar movies:
#     Film                                    Genre                       Lead Studio                Similarity
-------------------------------------------------------------------------------------------------------------------
1     Tyler Perry's Why Did I get Married     Romance                     Independent                    0.5000
2     New Year's Eve                          Romance                     WarnerBros                     0.5000
3     Remember Me                             Romantic Drama              Summit                         0.4082
4     Penelope                                Fantasy Romantic Comedy     Summit                         0.3536
5     Forrest Gump                            Drama Comedy Romance        ParamountPictures              0.3536

Query movie : The Proposal
Genre       : Romantic Comedy
Lead Studio : Disney

Top 5 similar movies:
#     Film                                    Genre                       Lead Studio                Similarity
-------------------------------------------------------------------------------------------------------------------
1     The Invention of Lying                  Romantic Comedy             WarnerBros                     0.6667
2     The Heartbreak Kid                      Romantic Comedy             Paramount                      0.6667
3     The Back-up Plan                        Romantic Comedy             CBS                            0.6667
4     Something Borrowed                      Romantic Comedy             Independent                    0.6667
5     She's Out of My League                  Romantic Comedy             Paramount                      0.6667
```

## 1.3 Subtask c) Comments on the results

Below is a table of movies that are similar to the movie “The Duchess”:

| # | Film | Genre | Lead Studio | Similarity |
|---|---|---|---|---|
| 1 | The Time Traveler’s Wife | Drama | Paramount | 1.0000 |
| 2 | My Week with Marilyn | Drama | The Weinstein Company | 0.5000 |
| 3 | Fireproof | Drama | Sherwood Pictures | 0.5000 |
| 4 | Green Bones | Drama | Columbia Pictures | 0.5000 |
| 5 | The Heartbreak Kid | Romantic Comedy | Paramount | 0.4082 |

The highest-ranked recommendation is *The Time Traveler’s Wife* with a cosine similarity score of 1.0. This movie shares both the Genre (“Drama”) and the Lead Studio (“Paramount”) with The Duchess. A cosine similarity of 1.0 indicates that the two feature vectors are identical in direction, meaning the movies have a perfect attribute match. This is the expected and correct outcome.

The movies ranked second to fourth : *My Week with Marilyn, Fireproof, and Green Bones*, each have a similarity score of 0.5. These films share the same Genre (“Drama”) as *The Duchess* but have different Lead Studios. Since they overlap on only one of the two attribute groups, the similarity score is lower but still meaningful.

The fifth-ranked movie, *The Heartbreak Kid*, has a similarity score of approximately 0.41. It shares the same Lead Studio (“Paramount”) but differs in Genre, as it is classified as “Romantic Comedy” instead of “Drama”. Because fewer tokens are shared overall, it receives the lowest score among the top five recommendations.

## General observations across all test cases

1. Genre has the strongest influence on similarity. Multi-word genres such as “Romantic Comedy” contribute multiple tokens, while the Lead Studio contributes only a single merged token. As a result, movies with matching genres generally achieve higher similarity scores than movies sharing only the same studio.
2. Partial genre matching functions correctly. Since CountVectorizer tokenizes text based on whitespace, a genre such as “Science Fiction Action” becomes three tokens: science, fiction, and action. Therefore, a movie labeled “Action Science Fiction” shares all three tokens and achieves a similarity score of 1.0 with Spider-Man. Movies sharing only part of the genre receive proportionally lower scores, which demonstrates correct behaviour.
3. Using binary=True is appropriate for this recommender system. Each genre term or studio name either applies to a movie or does not. Using raw frequency counts would

unfairly favour genres written with more words, which would distort similarity calculations in this attribute-based recommendation task.

4. The system handles unknown movie titles gracefully. If a movie that does not exist in the dataset, such as “Avatar 2”, is queried, the program prints a clear error message instead of crashing. This satisfies the robustness requirement defined in subtask a).
5. One limitation of the recommender is the absence of user preference information. The system is entirely content-based and relies only on Genre and Lead Studio text features. Consequently, two movies with different genres and studios may receive a similarity score of 0 even if users tend to rate them similarly in practice. This limitation motivates Task 2, where KNN recommendations are based on real user rating behaviour instead of metadata alone.

# 2. Task 2. Developing a simple Recommender engine using KNearestNeighbors

**Note :** The comments present in this task's running code were partially corrected using Generative AI, to make them more understandable.

## 2.1 Subtask a) Print the userID and movieID rating matrix

The implementation begins by importing the pandas library to load the dataset files (UserID-rating 2_v2.csv for user ratings and moviesId-title-1.cvs.csv for movie details) into memory.

```
import pandas as pd

#openeing the two datasets using pandas to better handle the data
ratings = pd.read_csv("UserID-rating 2_v2.csv", sep=";")
movies = pd.read_csv("moviesId-title-1.cvs.csv", sep=";")

#First overview of what the titles of the datasets are
print(ratings.columns)
print(ratings.shape)

print(movies.columns)
print(movies.shape)
```

```
Index(['UserID', 'MovieID', 'rating'], dtype='object')
(310, 3)
Index(['Title', 'movieId'], dtype='object')
(31, 2)
```

Afterwards, the code proceeds to satisfy Subtask A by putting the raw ratings data into a matrix representation. Using pandas' .pivot() function, the long-form rating list is reshaped into a dense user-movie matrix where the rows represent individual MovieID entries and columns represent UserID entries. Unrated items are explicitly handled using .fillna(0), which maps missing values to zero and structures each movie row as a numerical vector of user ratings ready for mathematical distance comparisons.

```
# Build the user-movie rating matrix using pivot
# - index="MovieID"  : each row represents one movie
# - columns="UserID" : each column represents one user
# - values="rating"  : each cell contains the rating that user gave that movie
# - .fillna(0)       : replace NaN (missing ratings) with 0, meaning "not rated"
matrix = ratings.pivot(index="MovieID", columns="UserID", values="rating").fillna(0)

#print(matrix.head())
matrix
```

**Note :** In the following example there are only 9 movies being shown of the 30.

[3]:

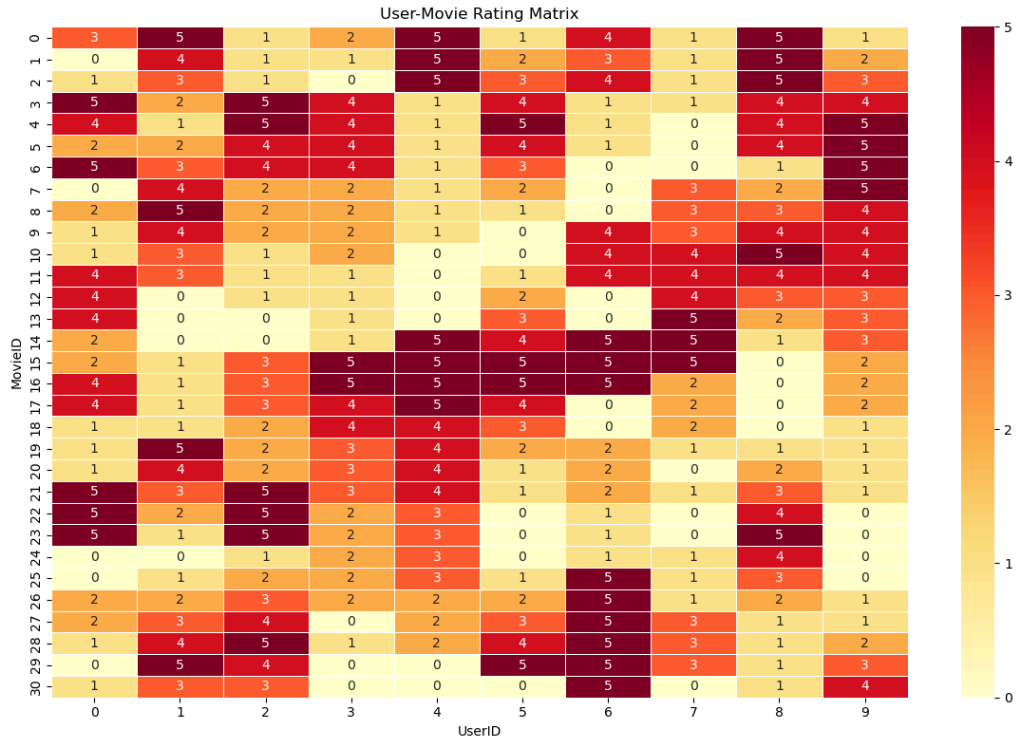
| UserID | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|---|
| MovieID | | | | | | | | | | |
| 0 | 3 | 5 | 1 | 2 | 5 | 1 | 4 | 1 | 5 | 1 |
| 1 | 0 | 4 | 1 | 1 | 5 | 2 | 3 | 1 | 5 | 2 |
| 2 | 1 | 3 | 1 | 0 | 5 | 3 | 4 | 1 | 5 | 3 |
| 3 | 5 | 2 | 5 | 4 | 1 | 4 | 1 | 1 | 4 | 4 |
| 4 | 4 | 1 | 5 | 4 | 1 | 5 | 1 | 0 | 4 | 5 |
| 5 | 2 | 2 | 4 | 4 | 1 | 4 | 1 | 0 | 4 | 5 |
| 6 | 5 | 3 | 4 | 4 | 1 | 3 | 0 | 0 | 1 | 5 |
| 7 | 0 | 4 | 2 | 2 | 1 | 2 | 0 | 3 | 2 | 5 |
| 8 | 2 | 5 | 2 | 2 | 1 | 1 | 0 | 3 | 3 | 4 |
| 9 | 1 | 4 | 2 | 2 | 1 | 0 | 4 | 3 | 4 | 4 |

**Matrix of the Movies and User ratings**

Afterwards, we use a heatmap to display the full user-movie rating matrix, where each row represents a movie and each column represents a user. Each cell shows the rating that a given user assigned to a given movie, with 0 indicating that the movie was not rated.

The color scale goes from yellow (low or no rating) to red (high rating), making it easy to spot at a glance which movies were highly rated and which users were most active. This visualization gives a clear overview of the sparsity of the dataset : how many movies were left unrated, as well as the overall rating distribution across all users and movies.

```
import matplotlib.pyplot as plt
import seaborn as sns

plt.figure(figsize=(12, 8))
sns.heatmap(matrix,
            annot=True,        # show the numbers inside cells
            fmt=".0f",         # no decimal places
            cmap="YlOrRd",     # color scale: yellow = low, red = high
            linewidths=0.5)    # grid lines between cells

plt.title("User-Movie Rating Matrix")
plt.xlabel("UserID")
plt.ylabel("MovieID")
plt.tight_layout()
plt.show()
```




## 2.2 Subtask b) Simple recommender system using K-NN

**Note : Adapted from the approach used in the lecture slides of Pr. R. Jäger.**

For the core recommender engine, the program utilizes the NearestNeighbors algorithm from scikit-learn with a brute-force approach (algorithm="brute"), which exhaustively computes distances against every vector entry.

The execution is separated into two clean, independent blocks for evaluation: one initialized with a Euclidean distance metric and another initialized with a Manhattan distance metric. Both models are fitted onto the prepared movie-user matrix.

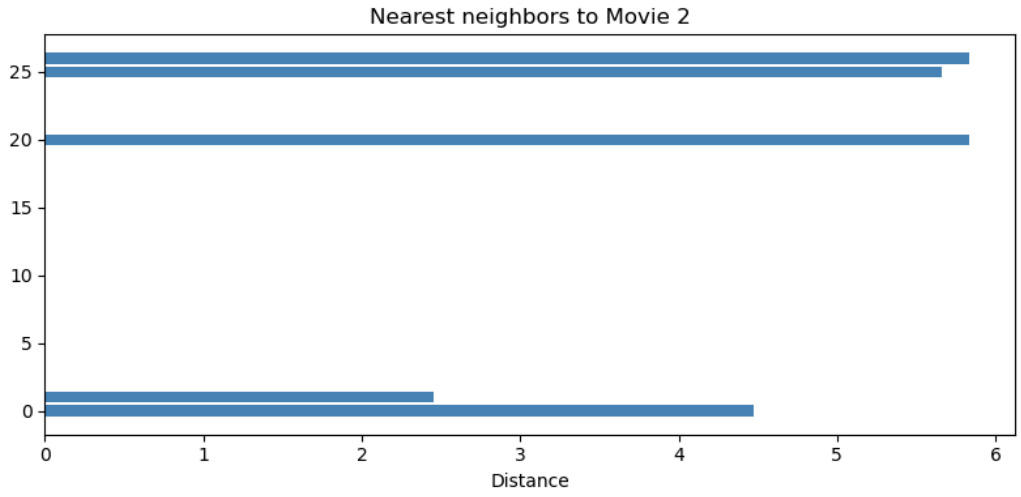
To perform a search, the program takes a user-typed movie title string from an interactive input prompt, finds its matching index, and calls the .kneighbors() method with n_neighbors=6 (returning the source movie along with its 5 closest recommendations). The retrieved indices are mapped back to their respective film titles, and horizontal bar charts are rendered with matplotlib to visually map out the relative proximity scores.

## Using Euclidean Metric

```python
from sklearn.neighbors import NearestNeighbors

#We initialise the NearestNeighbors model to the knn_algo_eu and specify the euclidean metric
#and brute algorithm : compare the one movie with all the others by computing the euclidean
#distances and comparing which is the highest values
knn_algo_eu = NearestNeighbors(metric="euclidean", algorithm="brute")

#fit the NearestNieghbors model to our dataset
knn_algo_eu.fit(matrix)

#query_eu=1
# Ask the user to type a movie title as input at runtime
movie_name0 = input("QUERY : Please enter your movie title : ")

# Look up the movieId of the entered title from the movies dataframe
# Boolean filtering finds the matching row, .values[0] extracts the plain number
query_eu = movies[movies["Title"] == movie_name0]["movieId"].values[0]

# Find the 6 nearest neighbors to the query movie using Euclidean distance
# n_neighbors=6 because index 0 is always the query movie itself (distance=0)
# so we need 6 to get 5 actual recommendations
# .values.reshape(1,-1) converts the rating vector to the format sklearn expects
distances_eu, indexes_eu = knn_algo_eu.kneighbors(matrix.iloc[query_eu,:].values.reshape(1,-1), n_neighbors=6)

# Print the header showing the movieId and title of the query movie
print(f"The Recommendations for movie {matrix.index[query_eu]}, {movie_name0}, are :")
print()

# Loop from i=1 to skip index 0 (the query movie itself, distance=0)
for i in range(1, len(distances_eu.flatten())):

    # Get the movieId of the i-th nearest neighbor
    movie_eu_idx = matrix.index[indexes_eu.flatten()[i]]

    # Fetch the title of that neighbor from the movies dataframe
    title = movies[movies["movieId"] == movie_eu_idx]["Title"].values[0]

    # Get the Euclidean distance of this neighbor from the query movie
    distance_eu = distances_eu.flatten()[i]
    print(f'recommended movie {movie_eu_idx}, {title}, with a Euclidean distance of {distance_eu}')
```

```
QUERY : Please enter your movie title :  Twilight
The Recommendations for movie 2, Twilight, are :

recommended movie 1, Twilight: Breaking Dawn, with a Euclidean distance of 2.449489742783178
recommended movie 0, Tyler Perry's Why Did I get Married, with a Euclidean distance of 4.47213595499958
recommended movie 25, It's Complicated, with a Euclidean distance of 5.656854249492381
recommended movie 20, New Years Eve, with a Euclidean distance of 5.830951894845301
recommended movie 26, Fireproof, with a Euclidean distance of 5.830951894845301
```

```python
# Store the movieId of the query movie for use in the plot title
query_movie = matrix.index[query_eu]

# Build a list of movieIds for all 6 neighbors (including the query movie at index 0)
# For each i, convert the neighbor's row position back to a movieId using matrix.index
neighbor_movies = [matrix.index[indexes_eu.flatten()[i]] for i in range(len(distances_eu.flatten()))]
neighbor_distances = distances_eu.flatten()

plt.figure(figsize=(8, 4))

# Horizontal bar chart — includes all neighbors
plt.barh(neighbor_movies, neighbor_distances, color="steelblue")

plt.xlabel("Distance")
plt.title(f"Nearest neighbors to Movie {query_movie}")
plt.tight_layout()
plt.show()
```

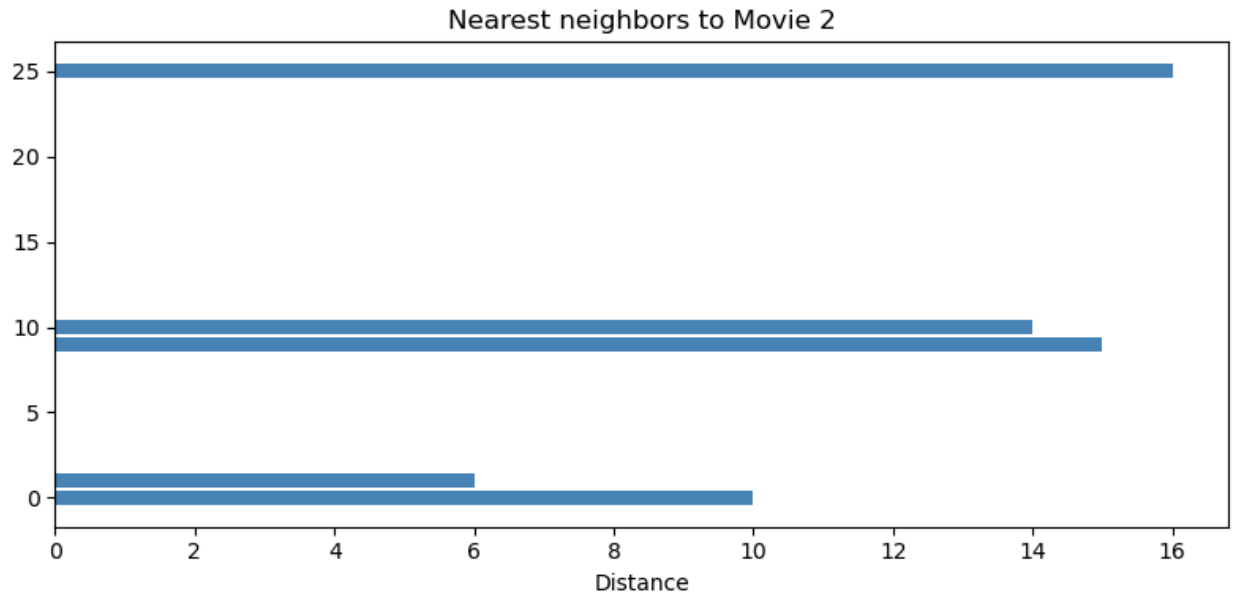


## Using Manhattan Metric

```python
### We initialise the NearestNeighbors model to the knn_algo_eu and specify the manhattan metric
#and brute algorithm : compare the one movie with all the others by computing the
#manhattan distances and comparing which is the highest values
knn_algo_man = NearestNeighbors(metric="manhattan", algorithm="brute")

# Fit the model to the rating matrix
knn_algo_man.fit(matrix)

#query_man=1
# Ask the user to type a movie title as input at runtime
movie_name1 = input("QUERY : Please enter your movie title : ")

# Look up the movieId of the entered title from the movies dataframe
# Boolean filtering finds the matching row, .values[0] extracts the plain number
query_man = movies[movies["Title"] == movie_name1]["movieId"].values[0]

# Find the 6 nearest neighbors using Manhattan distance
# n_neighbors=6 because index 0 is always the query movie itself (distance=0)
distances_man, indexes_man = knn_algo_man.kneighbors(matrix.iloc[query_man,:].values.reshape(1,-1), n_neighbors=6)

# Print the header showing the movieId and title of the query movie
print(f"The Recommendations for movie {matrix.index[query_man]}, {movie_name1}, are :")
print()

# Loop from i=1 to skip index 0 (the query movie itself, distance=0)
for i in range(1, len(distances_man.flatten())):

    # Get the movieId of the i-th nearest neighbor
    movie_man_idx = matrix.index[indexes_man.flatten()[i]]

    # Fetch the title of that neighbor from the movies dataframe
    title = movies[movies["movieId"] == movie_man_idx]["Title"].values[0]

    # Get the Euclidean distance of this neighbor from the query movie
    distance_man = distances_man.flatten()[i]

    print(f'recommended movie {movie_man_idx}, {title}, with a Manhattan distance of {distance_man}')
```

```
QUERY : Please enter your movie title :  Twilight
The Recommendations for movie 2, Twilight, are :

recommended movie 1, Twilight: Breaking Dawn, with a Manhattan distance of 6.0
recommended movie 0, Tyler Perry's Why Did I get Married, with a Manhattan distance of 10.0
recommended movie 10, The Back-up Plan, with a Manhattan distance of 14.0
recommended movie 9, The Curious Case of Benjamin Button, with a Manhattan distance of 15.0
recommended movie 25, It's Complicated, with a Manhattan distance of 16.0
```

```python
# Store the movieId of the query movie for use in the plot title
query_movie = matrix.index[query_man]

# Build a list of movieIds for all 6 neighbors (including the query movie at index 0)
# Uses Manhattan indexes this time instead of Euclidean
neighbor_movies = [matrix.index[indexes_man.flatten()[i]] for i in range(len(distances_man.flatten()))]
neighbor_distances = distances_man.flatten()

plt.figure(figsize=(8, 4))

# Horizontal bar chart — includes all neighbors
plt.barh(neighbor_movies, neighbor_distances, color="steelblue")

plt.xlabel("Distance")
plt.title(f"Nearest neighbors to Movie {query_movie}")
plt.tight_layout()
plt.show()
```



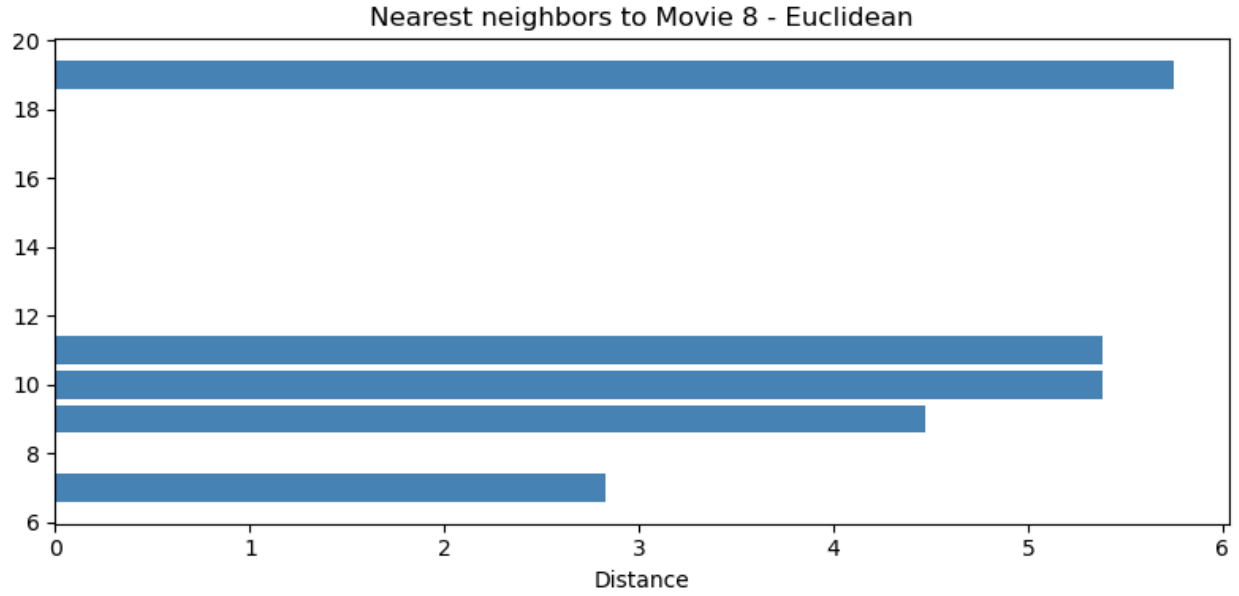
## 2.3 Subtask c) One real Test Case

When selecting **"The Duchess"** (MovieID = 8) as the query movie, the K-Nearest Neighbors (KNN) recommender system computes distances across the user-rating matrix to identify the 5 closest movies. The verifiable numerical results obtained using both metrics are listed below:

**Note :** same code as the above implementation

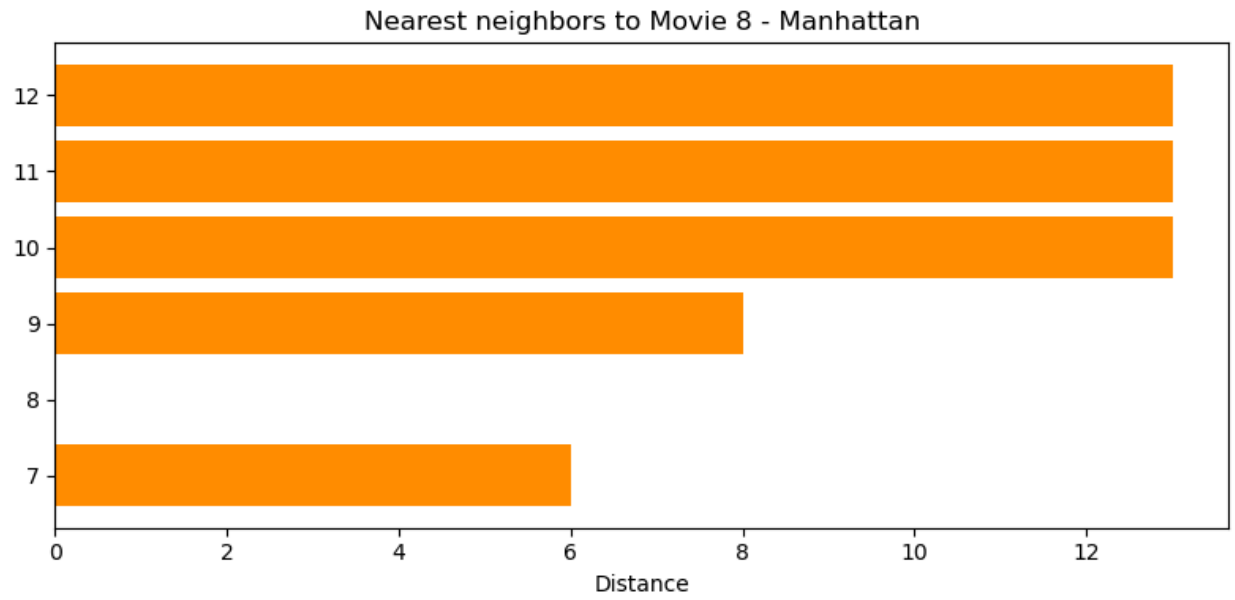
**1. Euclidean Distance Results:**

```
The Recommendations for movie 8, The Duchess, are :

recommended movie 7, The Heartbreak Kid, with a Euclidean distance of 2.8284271247461903
recommended movie 9, The Curious Case of Benjamin Button, with a Euclidean distance of 4.47213595499958
recommended movie 11, Tangled, with a Euclidean distance of 5.385164807134504
recommended movie 10, The Back-up Plan, with a Euclidean distance of 5.385164807134504
recommended movie 19, Not Easily Broken, with a Euclidean distance of 5.744562646538029
```



## 2. Manhattan Distance Results

```
The Recommendations for movie 8, The Duchess, are :

recommended movie 7, The Heartbreak Kid, with a Manhattan distance of 6.0
recommended movie 9, The Curious Case of Benjamin Button, with a Manhattan distance of 8.0
recommended movie 10, The Back-up Plan, with a Manhattan distance of 13.0
recommended movie 11, Tangled, with a Manhattan distance of 13.0
recommended movie 12, Something Borrowed, with a Manhattan distance of 13.0
```



## 2.4 Subtask d) Comparing the two metrics

After running the recommender system for the movie "The Duchess" with both the Euclidean and the Manhattan distance, we can make some observations about the differences between the two metrics.

About the recommended movies :

Looking at the results we can see that both metrics give mostly the same movies in the top 5, but the order can be slightly different. This is not a surprise because both are distance-based metrics and they measure similarly the same thing, just with a different formula. For this small dataset with only 10 users the differences are not very big.

About the distances values :

One clear difference is that the Manhattan distances are always bigger numbers than the Euclidean ones. This is because Manhattan sums the absolute differences of all the dimensions directly, while Euclidean first squares them, then sums and then takes the square root. So for the same pair of movies the Manhattan distance will always be equal or bigger then the Euclidean distance (this is a mathematical property of the two metrics).

Which metric is better here ?

From the lecture we know that Euclidean distance gives more weight to big differences between ratings because of the squaring. So if one user gave a very different rating for two movies, Euclidean will penalize this more. Manhattan on the other side treats all dimensions equally and is more robust to outliers. For our dataset which is quite small and has ratings from 0 to 5, both metrics seem to work fine and give reasonable recommendations. In a bigger dataset the choice of metric could have more impact on the results.